

INF201 – Lecture 02

14th Sept 2026

Today's topics

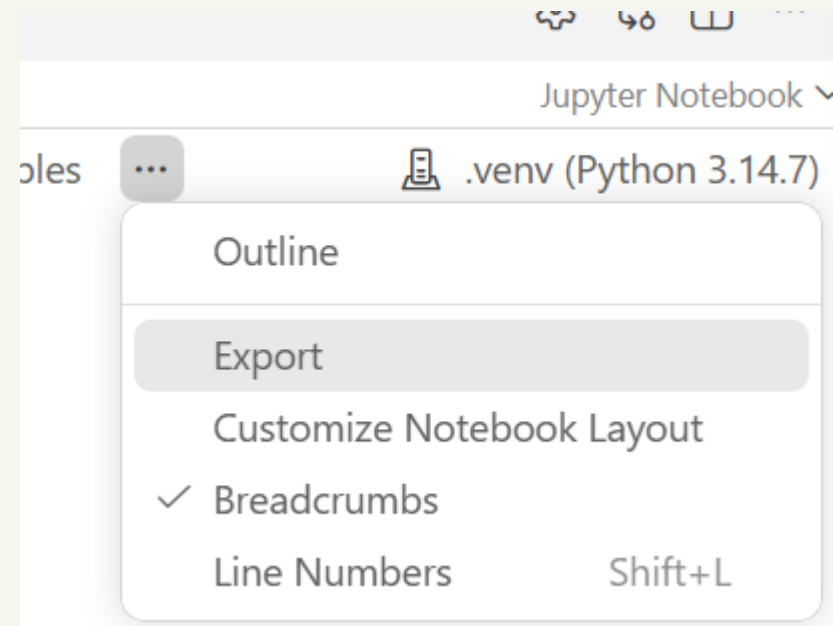
- Mandatory assignments
- Doc-strings
- String formatting
- Data types in Python
- Dynamic vs static typing
- Type hints
- A deep dive into lists

Mandatory assignments (week 37- 40)

- Attendance is mandatory, make sure the teaching assistant(s) register you
- You work in pre-assigned groups of 3-4 people, where you can discuss your code and learn from each other
- If the assignment text seems vague:
 - Use your creativity
 - Think about which assumptions you are making
- You will be rewarded up to 5 points per assignment (if and only if you attend the class)
- You need at least 13 points from these first four weeks

Mandatory assignments (week 37-40)

- You must upload your code (and AI chat-log if relevant) to Canvas by the deadline on Friday
 - You may submit this as a text file, Word (or similar) document or a pdf
 - Long chat log? You may hand in a summary instead
 - Suggested prompt: «Could you list the questions I have asked you and give a short summary of our conversation?»
- Even if you have worked together on your code, you must all upload *separately* on Canvas
- You can hand in python scripts (.py) or Jupyter notebooks (.ipynb)
 - If you use Jupyter notebook, please also submit a pdf version
 - You can export the notebook as pdf directly in VS Code (requires TeX)
 - Alternatively export it to html first, and then to pdf in your web browser
 - [Jupyter Notebooks in VS Code](#)
- Please do not hand in zipped folders



A quick chat about folder structure

- ▼ inf201
 - ▼ deliverable_1
 - > .venv
 - > .vscode
 - ▼ deliverable_2
 - > .venv
 - > .vscode
 - ▼ exercises
 - > .venv
 - > .vscode
 - > week_37
 - > week_38
 - > week_39
 - > week_40

Formatted string literals: f-strings

A very flexible way to format strings in Python - [PEP 498 – Literal String Interpolation](#)

```
# Formatting numbers in strings
number = 2
print(f"{number} divided by 3 is {number / 3}.")
print(f"{number} divided by 3 is {number / 3:.2f}.")
print(f"{number} divided by 3 is {number / 3:.1%}.")

# Alignment and padding
print(f"{42:0>5}")
print(f"{'Lower case':<15}{'Upper case':<15}")
for i in range(26):
    lower = chr(97 + i)
    upper = chr(65 + i)
    print(f"{lower:<15}{upper:<15}")
```

Doc-strings

- [PEP 257 – Docstring Conventions](#)
- Comments for documenting functions
- Starts on the first line after the function definition
- Triple quotes
- One-liners for fairly obvious definitions
- Can also list argument(s) and return(s)

```
def add_ints(x: int, y: int) -> int:  
    """Add two integers together."""  
    return x + y
```

```
def process_class_scores(scores: list[float], pass_threshold: float = 50.0) -> float:  
    """  
    Returns the average score of passing students.  
  
    Args:  
        scores: List of student scores.  
        pass_threshold: Score threshold for passing. Default is 50.0.  
    Returns:  
        Average score of passing students. Returns 0.0 if no students passed.  
    """
```

An (incomplete) list of Python types

- Numeric types
 - int, example: 3
 - float, example: 3.14
 - complex, example: 3+4j
- Sequence types
 - list, example: [1, "a", 3.14]
 - tuple, example: (1, "a", 3.14)
 - range, example: range(5)
- Text sequence types
 - string, example: "text"
- Set types
 - set, example: {1, "a", 3}
- Mapping types
 - dict, example: {"a" : 1, "b": 2}
- Special types
 - NoneType, example: None
- Boolean types
 - bool, example: True, False

[Built-in Types — Python 3.14.7 documentation](#) – for a more comprehensive list

Mutable vs immutable types

- Mutable: Can be changed in place

- Examples:

- list
- dict
- set

```
# Mutable
x = [1, 2, 3]
print(id(x))
x.append(4) # [1, 2, 3, 4]
print(id(x)) # the same as before
```

- Immutable: Cannot be changed in place

- Examples:

- int, float, complex
- string
- bool
- NoneType
- tuple*

```
# Immutable
s = "a b c"
print(id(s))
s += " d" # "a b c d"
print(id(s)) # changed
```



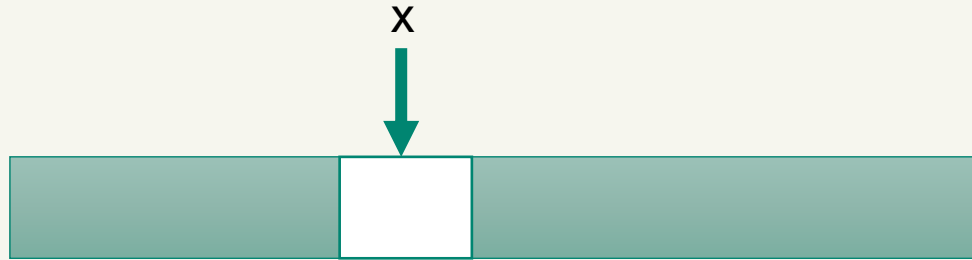
Python is dynamically typed

```
a = 3 # a is an integer  
a = "text" # a is now a string
```

- We don't have to tell Python which type a is supposed to be, it infers it
- When we change a, it can become a different type
- This would not be the case in a statically typed language

```
// C code  
int x = 4;
```

Dynamic vs static typing



Dynamic

- Example: Python, Ruby
- Type is determined at run-time
- A variable is a label or reference to an object in memory
- Faster and easier to write
- Unexpected type errors when running code

Static

- Example: C, Java
- Type is determined at compile-time
- Variables are typed “boxes” (locations) in memory
- Type errors are caught during compilation
- Code runs faster

Whirlwind Tour of Python has a good chapter on this ([Basic Python Semantics: Variables and Objects](#))



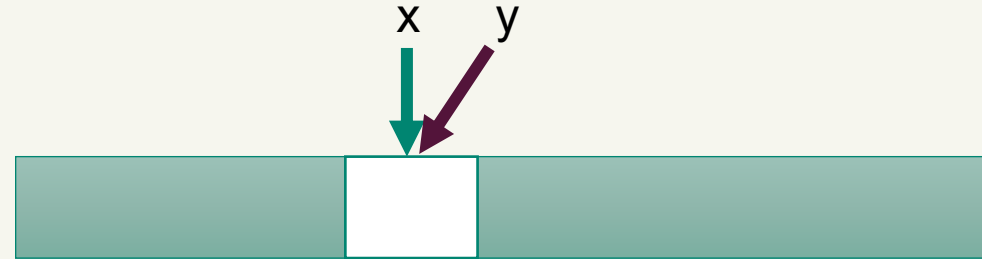
Beware of the aliasing trap

```
x = [1, 2, 3]
y = x
y.append(4)
print(y)  # Output: [1, 2, 3, 4]
print(x)
```

- [Python Tutor visualisation](#)
- [Facts and myths about Python names and values](#) (read this!)

Beware of the aliasing trap

```
x = [1, 2, 3]
y = x
y.append(4)
print(y)  # Output: [1, 2, 3, 4]
print(x)
print(id(x))  # Output: 1499266529280 (memory address)
print(id(y))  # Output: 1499266529280 (the same memory address)
print(id(x) == id(y))  # Output: True
```



- x and y point to the same object, as shown by their identical memory address
- x and y are aliases to the same object in memory
- Changing y therefore changes x as well
- [Python Tutor visualisation](#)
- [Facts and myths about Python names and values](#) (read this!)

Type hints

```
def process_class_scores(scores: list[float], pass_threshold: float = 50.0) -> float:
```

Type hints

```
def process_class_scores(scores: list[float], pass_threshold: float = 50.0) -> float:
```

- Type hints are a way to help avoid typing errors in Python
- Hints about which data type we expect the function input and output to have
- `list[float]` indicates that we expect the input score to be a list of floating-point numbers
- `-> float` indicates that we expect the function to return a floating-point number
- Warning: These are only hints, they will not be enforced
- We should still use type hints when you write functions!
- They help document the function and forces you to think this through when writing the function
- Type checkers like mypy will help you keep track of types
- You can read more about type hints here:
 - [PEP 484](#)
 - Chapter 2 of [Advanced Guide to Python 3 Programming](#)

Common type hint patterns

- Single type

```
def add_ints(x: int, y: int) -> int:  
    """Add two integers together."""  
    return x + y
```

- Multiple types

```
def add_with_hints(x: int | float, y: int | float) -> int | float:  
    """Add two numbers together."""  
    return x + y
```

- Containers

```
def calculate_grade_stats(  
    student_list: list[dict[str, str]], pass_threshold: float = 50.0  
) -> dict[str, Any]:
```

- Special cases: None, Any (from the typing module)

```
def my_print_function(x: int | float | str) -> None:  
    """Print a value."""  
    print(x)
```

A closer look at lists

- A list is a very useful *container* in Python
- A container can hold and structure a collection of other objects
- Sets, tuples and dicts are some other examples of containers
- A list is a *sequence*, meaning that the order of elements matter
- It is *random access*; you can access any element you want in any order
- It is *mutable*, meaning that is can be changed in place



Copying lists

- As we saw earlier, list2 is not a copy of list1
- They just both point to the same object

```
list1 = [1, 2, 3, 4, 5]  
list2 = list1
```

- We can use list1.copy() or list1[:] to create a shallow copy
- For nested lists, import deepcopy from copy
- [Visualisation in Python Tutor](#)

List comprehension

- We saw an example of this in last weeks' assignment

```
passing_scores = []  
for score in scores:  
    if score >= pass_threshold:  
        passing_scores.append(score)
```

```
passing_scores = [score for score in scores if score >= pass_threshold]
```

- A concise way to generate lists in Python
- Faster than using for loops
- Use for simple and moderately complex list generation
- Avoid for more complex cases, where the list comprehension becomes difficult to decode
- [List Comprehensions | A Whirlwind Tour of Python](#)